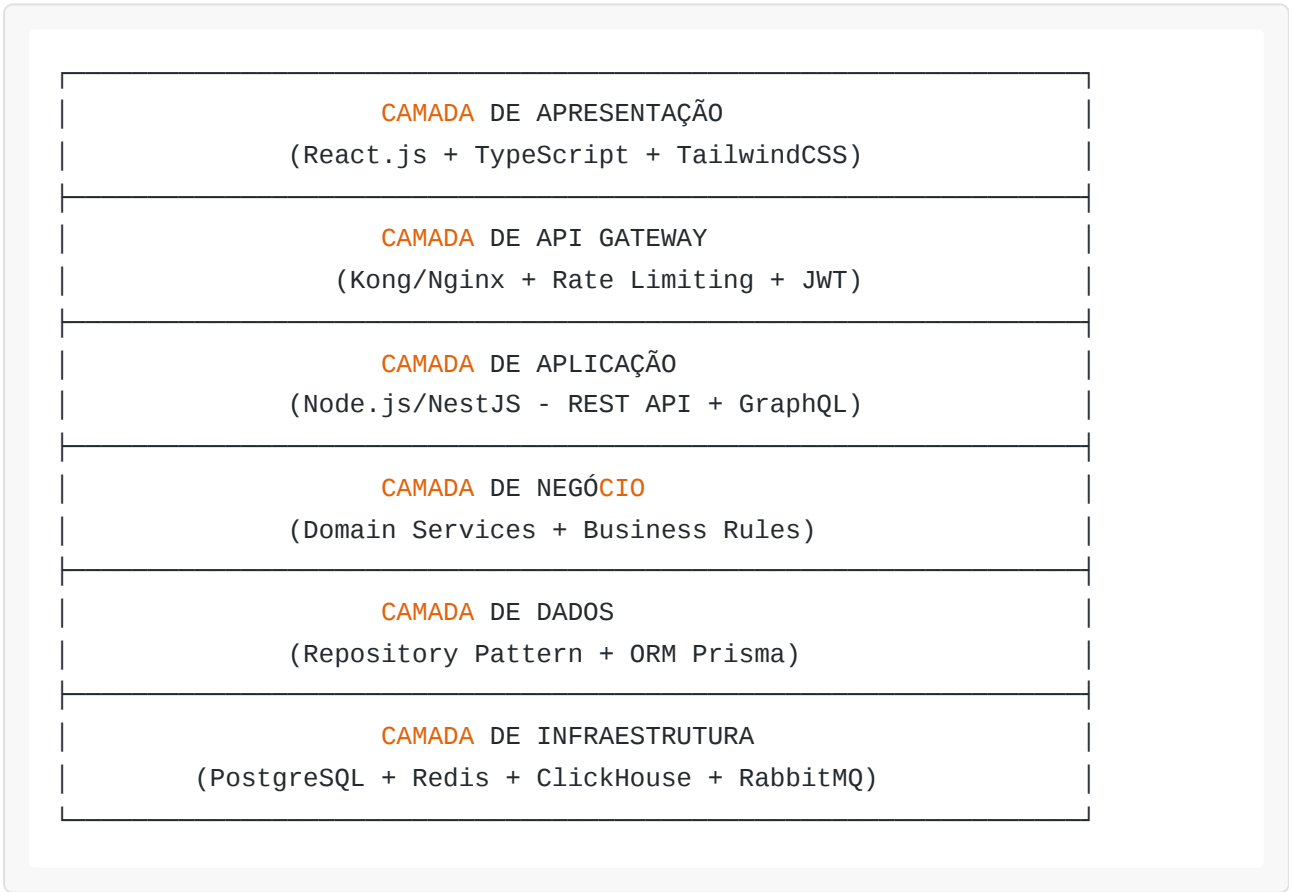
Arquitetura de Software - Sistema S&OP

Documento de Arquitetura Técnica

1. Visão Geral da Arquitetura

1.1 Estilo Arquitetural

O sistema S&OP será desenvolvido seguindo uma **arquitetura em camadas** com **microsserviços**, permitindo escalabilidade, manutenibilidade e evolução independente dos componentes.



1.2 Princípios de Design

Princípio	Descrição
SOLID	Aplicação dos princípios de design orientado a objetos
DDD	Domain-Driven Design para modelagem do domínio S&OP
CQRS	Separação de comandos e consultas para otimização
Event Sourcing	Rastreabilidade de mudanças em dados críticos
12-Factor App	Boas práticas para aplicações cloud-native

---

2. Stack Tecnológica

2.1 Frontend

Tecnologia	Versão	Propósito
React.js	18.x	Framework de UI
TypeScript	5.x	Tipagem estática
TailwindCSS	3.x	Estilização
Zustand	4.x	Gerenciamento de estado
React Query	5.x	Cache e sincronização de dados
ApexCharts	3.x	Gráficos e visualizações
AG-Grid	31.x	Tabelas avançadas
React Hook Form	7.x	Formulários
Zod	3.x	Validação de schemas

2.2 Backend

Tecnologia	Versão	Propósito
Node.js	20.x LTS	Runtime
NestJS	10.x	Framework backend
TypeScript	5.x	Tipagem estática
Prisma	5.x	ORM
Passport.js	0.7.x	Autenticação
class-validator	0.14.x	Validação
Bull	4.x	Filas de processamento

2.3 ML Service (Previsão de Demanda)

Tecnologia	Versão	Propósito
Python	3.11	Runtime
FastAPI	0.109.x	API REST
Pandas	2.x	Manipulação de dados
Scikit-learn	1.4.x	Modelos ML
Prophet	1.1.x	Previsão de séries temporais
Statsmodels	0.14.x	SARIMA, Holt-Winters
XGBoost	2.x	Gradient Boosting

2.4 Banco de Dados

Tecnologia	Propósito
PostgreSQL 16	Banco principal (OLTP)
Redis 7	Cache e sessões
ClickHouse	Analytics e histórico (OLAP)
MinIO/S3	Armazenamento de arquivos

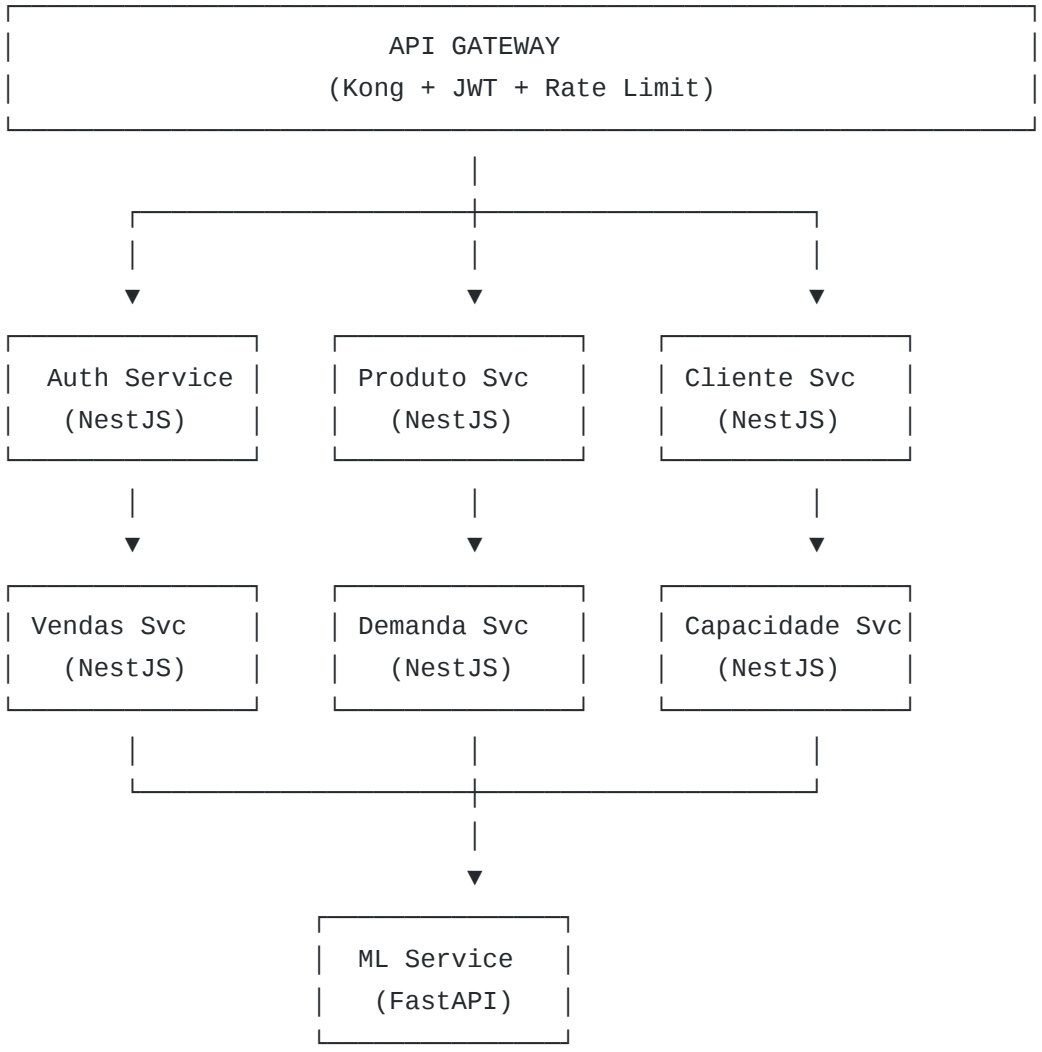
2.5 Infraestrutura

Tecnologia	Propósito
Docker	Containerização
Kubernetes	Orquestração
Kong	API Gateway
RabbitMQ	Message Broker
Prometheus	Métricas
Grafana	Dashboards de monitoramento
ELK Stack	Logs centralizados

---

3. Arquitetura de Microsserviços

3.1 Serviços Principais



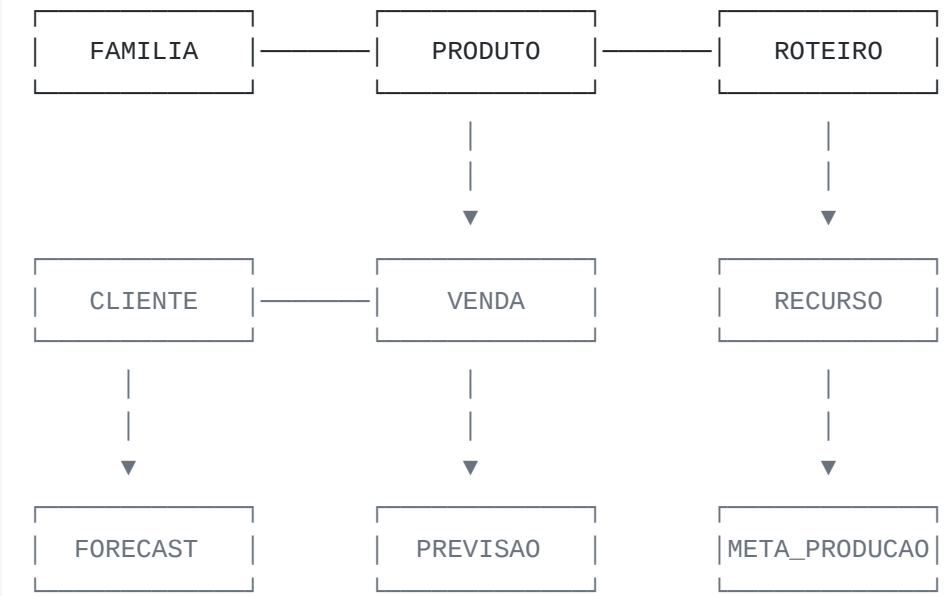
3.2 Responsabilidades dos Serviços

Serviço	Responsabilidades
Auth Service	Autenticação, autorização, gestão de usuários, JWT
Produto Service	CRUD de produtos, famílias, subfamílias
Cliente Service	CRUD de clientes, relacionamentos matriz/filial
Vendas Service	Histórico de vendas, carteira de pedidos, forecast
Demanda Service	Colaborações, consenso, acuracidade, previsões
Capacidade Service	Recursos, dias úteis, roteiros, IROG
ML Service	Modelos estatísticos de previsão

---

4. Modelo de Dados

4.1 Diagrama ER Simplificado



4.2 Schemas Principais

Produto

```
interface Produto {
  id: UUID;
  codigo: string;
  nome: string;
  familiaId: UUID;
  subfamiliaId: UUID;
  modelo: string;
  status: 'ATIVO' | 'INATIVO';
  custoTipo: 'MTS' | 'MTO';
  rateio: boolean;
  createdAt: Date;
  updatedAt: Date;
}
```

Cliente

```
interface Cliente {
  id: UUID;
  razaoSocial: string;
  cnpjCpf: string;
  clienteRelacionadoId?: UUID;
  status: 'ATIVO' | 'INATIVO';
  createdAt: Date;
}
```

Venda

```
interface Venda {
  id: UUID;
  nfNumero: string;
  sequencia: number;
  dataNf: Date;
  quantidade: Decimal;
  receitaBruta: Decimal;
  produtoId: UUID;
  clienteId: UUID;
  membroId: UUID;
  canalVenda: string;
  forcaVenda: string;
  zonaVenda: string;
  codigoPedido: string;
  createdAt: Date;
}
```

Previsao

```
interface Previsao {
  id: UUID;
  produtoId: UUID;
  ano: number;
  mes: number;
  valorPrevisto: Decimal;
  modeloUtilizado: string;
  mape: Decimal;
  wmape: Decimal;
  createdAt: Date;
}
```

---

5. APIs e Endpoints

5.1 Estrutura de Endpoints

```
/api/v1
├── /auth
│   ├── POST /login
│   ├── POST /logout
│   ├── POST /refresh
│   └── GET /me
│
├── /produtos
│   ├── GET /
│   ├── GET /:id
│   ├── POST /
│   ├── PUT /:id
│   ├── DELETE /:id
│   └── POST /import
│
├── /clientes
│   ├── GET /
│   ├── GET /:id
│   ├── POST /
│   ├── PUT /:id
│   └── POST /import
│
├── /vendas
│   ├── GET /
│   ├── GET /dashboard
│   ├── POST /import
│   └── GET /historico
│
├── /demanda
│   ├── GET /dashboard
│   ├── GET /colaboracoes
│   ├── POST /colaboracoes/:id/aprovar
│   ├── GET /consensos
│   ├── POST /consensos/:id/aprovar
│   ├── GET /acuracidade
│   └── GET /previsoes
│
├── /capacidade
│   ├── GET /dashboard
│   ├── GET /recursos
```

```
│   ├── POST /recursos
│   ├── GET /dias-uteis
│   ├── PUT /dias-uteis
│   ├── GET /roteiros
│   └── POST /roteiros
│
└── /producao
    ├── GET /dashboard
    ├── POST /lancamento-diario
    ├── GET /paradas
    ├── POST /paradas
    └── GET /metas
```

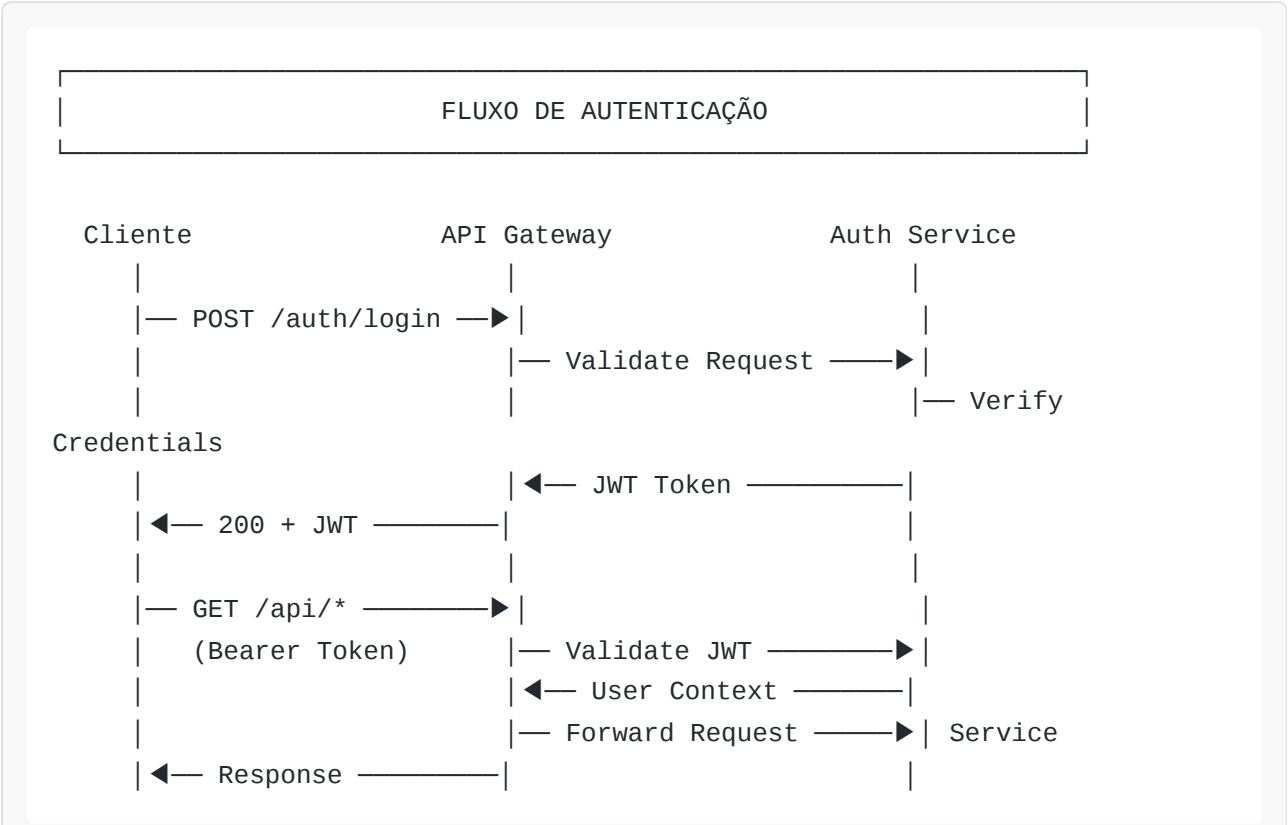
5.2 Padrão de Resposta

```typescript
interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: {
    code: string;
    message: string;
    details?: any;
  };
  meta?: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
  };
}
```

---

6. Segurança

6.1 Autenticação e Autorização



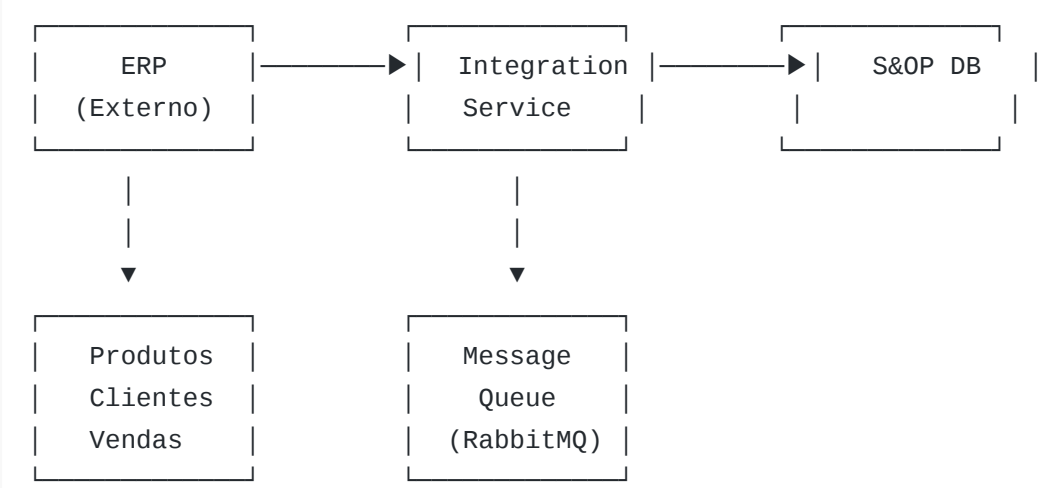
6.2 Perfis de Acesso

Perfil	Permissões
Admin	Acesso total ao sistema
Planejamento	Demanda, Capacidade, Consenso
Comercial	Vendas, Forecast, Colaboração Comercial
Marketing	Colaboração Marketing, Relatórios
Operações	Produção, Recursos, Paradas
Visualizador	Apenas leitura de dashboards

---

7. Integrações

7.1 Integração com ERP



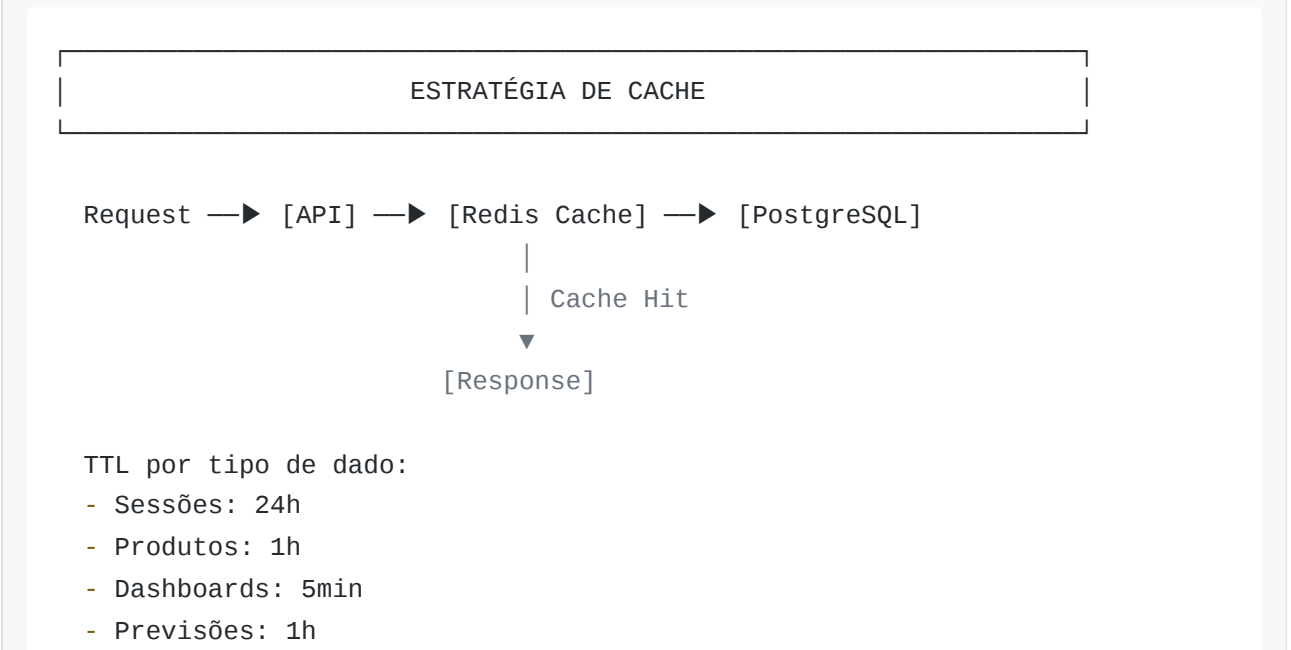
7.2 Métodos de Integração

Método	Uso
REST API	Consultas em tempo real
Webhook	Notificações de eventos
File Import	Importação em lote (CSV/Excel)
Message Queue	Processamento assíncrono

---

8. Escalabilidade e Performance

8.1 Estratégias de Cache



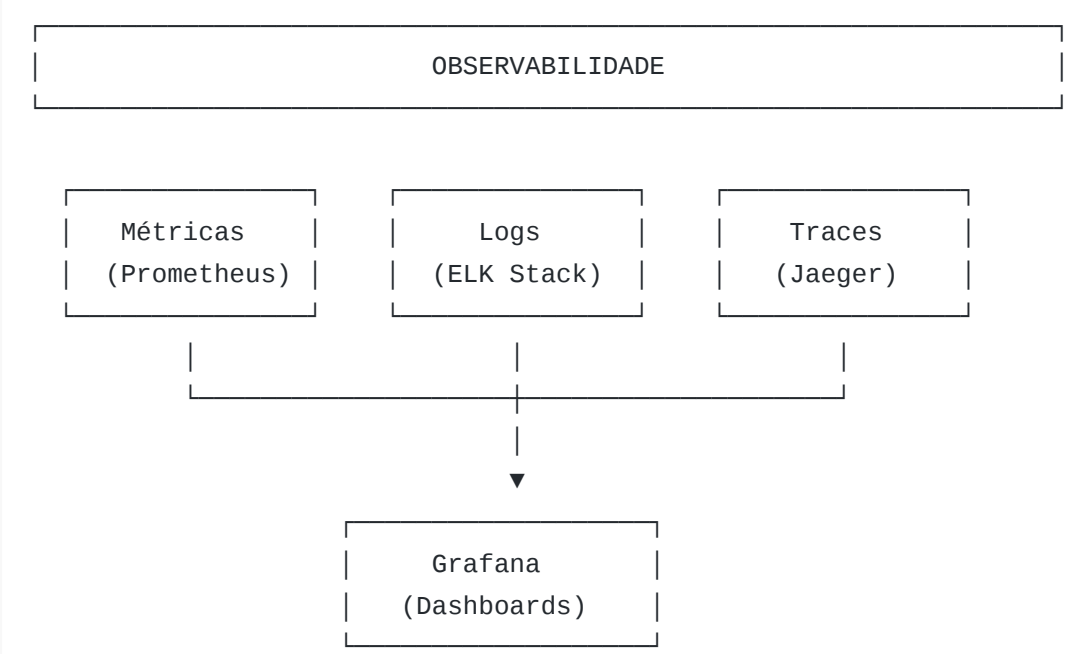
8.2 Otimizações de Banco

Estratégia	Aplicação
Índices	Campos de busca frequente
Particionamento	Tabelas de vendas por ano/mês
Read Replicas	Consultas de relatórios
Connection Pooling	PgBouncer
CQRS	Separação de leitura/escrita

---

9. Monitoramento e Observabilidade

9.1 Stack de Monitoramento



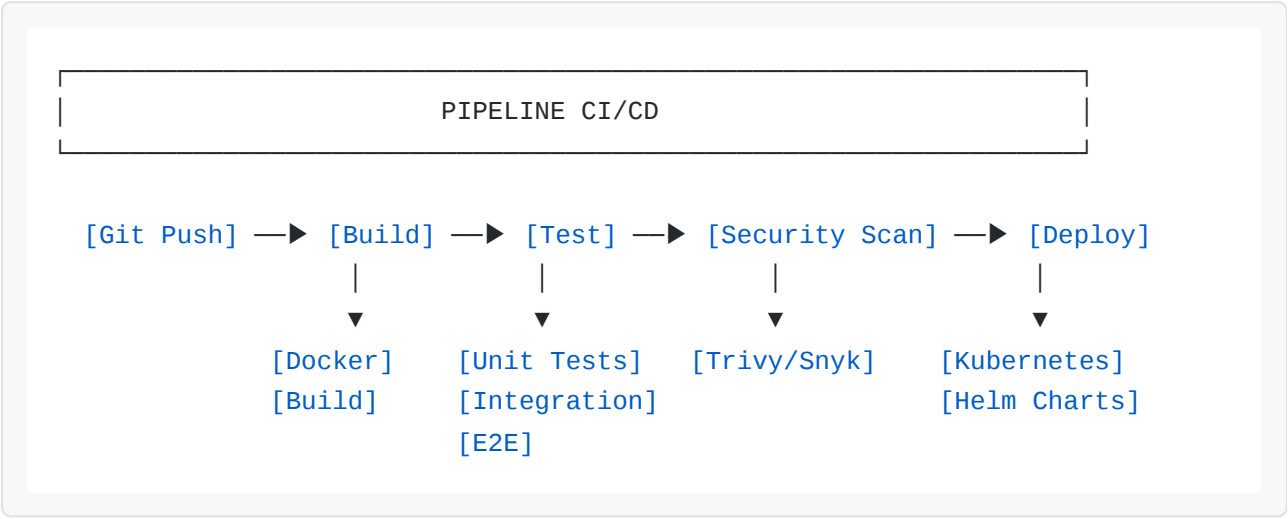
9.2 Métricas Principais

Métrica	Descrição	Alerta
Response Time	Tempo de resposta da API	> 500ms
Error Rate	Taxa de erros 5xx	> 1%
CPU Usage	Uso de CPU dos containers	> 80%
Memory Usage	Uso de memória	> 85%
DB Connections	Conexões ativas	> 80% pool
Queue Size	Tamanho das filas	> 1000 msgs

---

10. Deployment

10.1 Pipeline CI/CD



10.2 Ambientes

Ambiente	Propósito	Infraestrutura
Development	Desenvolvimento local	Docker Compose
Staging	Testes e homologação	Kubernetes (1 réplica)
Production	Produção	Kubernetes (3+ réplicas)

---

11. Estimativa de Recursos

11.1 Infraestrutura Mínima (Produção)

Componente	Especificação	Quantidade
API Servers	2 vCPU, 4GB RAM	3
ML Service	4 vCPU, 8GB RAM	2
PostgreSQL	4 vCPU, 16GB RAM, 500GB SSD	1 Primary + 1 Replica
Redis	2 vCPU, 4GB RAM	1
ClickHouse	4 vCPU, 16GB RAM, 1TB SSD	1

11.2 Custo Estimado (Cloud)

Provedor	Custo Mensal Estimado
AWS	~1.500−2.500
Azure	~1.400−2.300
GCP	~1.300−2.200

---

12. Roadmap de Implementação

Fase 1: MVP (3-4 meses)

- [ ] Setup de infraestrutura básica
- [ ] Auth Service
- [ ] Produto Service
- [ ] Cliente Service
- [ ] Frontend básico com cadastros

Fase 2: Core S&OP (3-4 meses)

- [ ] Vendas Service
- [ ] Demanda Service (Colaborações, Consenso)
- [ ] Dashboards de Demanda e Vendas
- [ ] Importação de dados

Fase 3: Capacidade e Produção (3-4 meses)

- [ ] Capacidade Service
- [ ] ML Service (Modelos de Previsão)
- [ ] Dashboard de Capacidade
- [ ] Dashboard de Produção

Fase 4: Otimização (2-3 meses)

- [ ] Integrações com ERP
- [ ] Relatórios avançados
- [ ] Alertas e notificações
- [ ] Otimização de performance

---

13. Considerações Finais

13.1 Pontos de Atenção

1. **Qualidade de Dados**: O sucesso do S&OP depende da qualidade dos dados de entrada
2. **Adoção do Usuário**: Interface intuitiva e treinamento adequado
3. **Performance**: Dashboards devem carregar em menos de 3 segundos
4. **Segurança**: Dados sensíveis de vendas e clientes

13.2 Próximos Passos

1. Validar arquitetura com stakeholders
2. Definir equipe de desenvolvimento
3. Configurar ambiente de desenvolvimento
4. Iniciar sprint 0 (setup de infraestrutura)